

SIAGA: Real-World Readiness Assessment

An honest evaluation of what SIAGA can and cannot do today, and what stands between the current build and a system a health ministry could run in production. Written to be truthful rather than promotional, because overstating readiness is how pilots fail.

1. The direct answer

Can SIAGA be used in the real world right now?

As a decision-support and prioritisation tool: yes, today, with a human in the loop. A regional health analyst could use SIAGA now to rank the drivers of life expectancy, compare member states, stress-test a budget scenario, and see five-year disease forecasts. The model is validated out-of-source (within 2.3 years against World Bank data it never saw), the recommendations are grounded in both the data and established WHO/UN practice, and the system runs as deployable containers with authentication and rate limiting.

As an autonomous or clinical decision-maker: no, and it should never be marketed as one. SIAGA operates on aggregate national indicators, not individual patients. Its predictions carry a real error bar (roughly plus or minus 1 to 3 years depending on the task) and rest on a small panel of 10 countries. It informs human judgment; it does not replace it.

This is the honest posture: **a production-ready decision-support prototype, not a simulation, and not an oracle.**

2. What is genuinely production-grade today

Capability	Status	Evidence
Reproducible training pipeline	Done	<code>python -m pipeline.train</code> runs ingest to deploy from one command
Model versioning and lineage	Done	<code>models/registry.json</code> , model card, metric history per run
Out-of-source validation	Done	scored against World Bank 2015-2019, a different source
Explainability for non-technical users	Done	SHAP driver ranking in years of life expectancy
API authentication and rate limiting	Done	API-key auth, per-IP token-bucket limiter
Containerised deployment	Done	Dockerfiles + compose for API and dashboard
Graceful degradation offline	Done	API is a static binary; pipeline falls back to cached data
Honest uncertainty	Done	forecast intervals, backtested model selection, stated limits

This is what makes it "not just a simulation": the model is trained by a real pipeline, versioned, served over a secured API, and consumed by a dashboard, all reproducibly.

3. What is NOT yet production-grade (the honest gaps)

1. **Data volume.** Ten countries and eleven years is a small panel. The model generalises acceptably, but a production system would ingest the full World Bank and WHO history (all countries, 2000 to present) to train on thousands of rows instead of 98. The pipeline is already built to absorb this; it is a data-ingestion task, not a rearchitecture.
 2. **Automated retraining and monitoring.** Retraining is currently a manual command. Production needs a schedule (for example monthly), plus drift detection that alerts when incoming data diverges from the training distribution.
 3. **Data freshness and provenance.** SIAGA uses annual historical indicators. Real early warning needs near-real-time feeds (surveillance reports, climate, mobility), with the blockchain consent and audit layer from the roadmap for cross-border trust.
 4. **Formal validation with domain experts.** The recommendations are evidence-aligned, but before influencing a real budget they need sign-off from epidemiologists and the relevant health ministries.
 5. **Security hardening.** Auth and rate limiting exist; production would add TLS, secret management, audit logging, and a penetration test.
 6. **Model governance.** A production health model needs a documented approval process, bias and equity audits across countries, and a rollback plan (the registry is the foundation for this).
-

4. The continuous-improvement pipeline (how SIAGA gets better over time)

SIAGA is built as a loop, not a one-off artifact:

```
data/raw + World Bank API
|
v
pipeline: ingest -> clean -> features -> train -> validate -> explain -> forecast
|
v
models/ registry (versioned model + card + metrics history)
|
v
serving files -> Go API (auth, rate limit) -> Streamlit dashboard
|
v
new data arrives -> back to the top
```

To improve the model, a maintainer:

1. Drops new or corrected data into `data/raw` (or lets the World Bank pull refresh).
2. Runs `python -m pipeline.train --version vN`.
3. Reviews the new metrics in `models/registry.json` against the previous version.
4. If better, the serving files and deployed model update automatically; if worse, the previous version is still recorded and can be restored.

This is the mechanism that turns SIAGA from a hackathon submission into a system that compounds in value as more data and feedback arrive.

5. Recommended next 90 days (if this became a funded pilot)

- 1. **Weeks 1-3:** ingest full World Bank and WHO history; retrain and re-validate on the larger panel. Expect the unseen-country error to fall.
 - 2. **Weeks 4-6:** add scheduled retraining and drift monitoring; wire TLS and audit logging onto the API.
 - 3. **Weeks 7-9:** expert review of drivers and recommendations with one partner ministry; incorporate feedback.
 - 4. **Weeks 10-12:** a live pilot in one country with a real analyst, measuring whether SIAGA changed a real allocation decision. That is the only test that ultimately matters.
-

6. Bottom line

SIAGA today is a credible, honest, deployable decision-support tool with a real MLOps loop behind it. It is good enough to inform real prioritisation now, with a human in the loop, and it is architected so that more data and a short hardening phase move it toward production without a rewrite. It is deliberately not sold as a finished clinical system, because the fastest way to lose a health ministry's trust is to overclaim.

